Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра автоматизированных систем управления (АСУ)

ПОИСК МИНИМУМА ФУНКЦИИ ОДНОЙ ПЕРЕМЕННОЙ НА МНОЖЕСТВЕ ЦЕЛЫХ ЧИСЕЛ

Отчёт по лабораторной работе №2
по дисциплине
«Эвристические методы оптимизации»

Студент гр. 435-М1
_____ Д.П. Андреев
«__» _____ 2025

Проверил
Профессор каф. АСУ, д.т.н
_____ А.А. Мицель
«__» _____ 2025

Томск 2025

Оглавление

Введение 3
1 Краткая теория 4
2 Ход работы 6
Вывод 13

Введение

Цель работы: научиться вычислять приближенное решение задачи одномерной оптимизации на множестве целых чисел с помощью генетических алгоритмов.

Задание на лабораторную работу:

1. Сгенерировать начальную популяцию бинарных хромосом.
2. Декодировать их в решения и оценить приспособленность.
3. Отобрать родителей с вероятностью, зависящей от приспособленности.
4. Скрестить выбранные пары, получив потомков.
5. Применить мутации с заданной вероятностью.
6. Сохранить лучших особей (элитарный отбор).
7. Повторять шаги 4–6 до формирования новой популяции.
8. Оценить приспособленность новой популяции.
9. Проверить критерий остановки; если не выполнен — начать новый цикл.

Входные данные, согласно варианту 1, следующие:

1. Целевая функция: $f(x) = 2x^2 + 1$.
2. Границы интервала: $x \in [-31; 31]$.
3. Количество особей в популяции К = 10.
4. Количество поколений N = 5.
5. Параметр окончания поиска: $\varepsilon = 0{,}001$

1 КРАТКАЯ ТЕОРИЯ

Ген – атомарный элемент хромосомы (может быть битом, числом или неким другим объектом.

Хромосома (цепочка) – упорядоченная последовательность генов (строка из каких-либо чисел). Если эта строка представлена бинарной строкой из нулей и единиц, например, 101010, то она получена либо с использованием двоичного кодирования, либо кода Грея. Каждая позиция хромосомы называется геном.

Генотип (код) – упорядоченная последовательность хромосом (одно из решений).

Фенотип – набор значений, соответствующих генотипу.

Особь (индивидуум) – конкретный экземпляр генотипа (вариант решения задачи). Особи представляются хромосомами с закодированными в них множествами параметров задачи, т.е. решений. Обычно особь состоит из одной хромосомы, поэтому в дальнейшем особь и хромосома идентичные понятия.

Популяция – набор особей (набор решений задачи). В начале алгоритма случайным образом генерируется начальная популяция (набор решений). Эти решения будут становиться лучше (эволюционировать) в процессе работы алгоритма до тех пор, пока не удовлетворят условиям задачи.

Кроссинговер (кроссовер) – операция, при которой две хромосомы обмениваются своими частями.

Мутация – случайное изменение одной или нескольких позиций в хромосоме.

Инверсия – изменение порядка следования битов в хромосоме или в ее фрагменте.

Рекомбинация – операция, при которой две хромосомы обмениваются своими частями.

Функция приспособленности – оценивает то, насколько приспособлена данная особь в популяции, другими словами, она определяет качество особей популяции.

2 ХОД РАБОТЫ

Для начала нам необходимо сформировать популяцию из 10 особей и закодировать значения переменной в виде двоичных последовательностей. На рисунке 2.1 представлен текст программы, формирующей начальную популяцию на множестве целых чисел $x \in [-31; 31]$

```
init_population(a, b, count) := | for elem ∈ 1..count
                                |   out_elem ← a + floor(rnd(b − a + 1))
                                | out
```

Рисунок 2.1 – Текст программы

Затем при помощи метода рулетки выберем K родителей для создания новой популяции. На рисунке 2.2 представлен текст программы.

```
ruletka(p) := | rows ← rows(p)
              | for row ∈ 1..rows
              |   | dart ← rnd(100)
              |   | sector ← 0
              |   | for elem ∈ 1..rows
              |   |   | sector ← sector + p_elem
              |   |   | if sector ≥ dart
              |   |   |   | out_row ← elem
              |   |   |   | break
              | out
```

Рисунок 2.2 – Текст программы

Для метода рулетки необходимо рассчитать веса. Расчет весов осуществляется программой, текст которой представлен на рисунке 2.3.

```
weight(f, x) := | K ← rows(x)
                | denominatior ← Σ_{i=1}^{K} |f(x_i)|
                | for i ∈ 1..K
                |   out_i ← |f(x_i)| / denominatior
                | out
```

Рисунок 2.3 – Текст программы

Следующим шагом будет выбор из родительского пула пары родителей для репродукции, благодаря оператору кроссинговера получим потомка. На рисунке 2.4 представлен текст программы генетического оператора Кроссинговера.

```
crosingover(parents, a, b) := | cod2_parents ← decimalVector_to_cod2Matrix(parents)
                              | rows ← rows(cod2_parents)
                              | cols ← cols(cod2_parents)
                              | inverted_cod2_Parents ← cod2_parents^T
                              | for row ∈ 1..round(rows/2)
                              |   | changing_index ← floor(1 + rnd(cols))
                              |   | firstRow ← row·2 − 1
                              |   | secondRow ← mod[firstRow + 1, rows + (mod(rows + 1, 2))]
                              |   | out ← insertVectorToMatrix(out, inverted_cod2_Parents^⟨firstRow⟩)
                              |   | out_{firstRow, changing_index} ← cod2_parents_{secondRow, changing_index}
                              |   | if secondRow > firstRow
                              |   |   | out ← insertVectorToMatrix(out, inverted_cod2_Parents^⟨secondRow⟩)
                              |   |   | out_{secondRow, changing_index} ← cod2_parents_{firstRow, changing_index}
                              |   | if out_{firstRow, changing_index}·out_{secondRow, changing_index} = −1
                              |   |   | out_{firstRow, changing_index} ← 0
                              |   |   | out_{secondRow, changing_index} ← 0 if secondRow > firstRow
                              | out ← cod2Matrix_to_decimalVector(out)
                              | for row ∈ 1..rows
                              |   | if out_row < a
                              |   |   | out_row ← a
                              |   |   | continue
                              |   | out ← b if out_row > b
                              | out
```

Рисунок 2.4 – Текст программы

Подвергнем потомков мутации, благодаря программе на рисунке 2.5.

```
mut(population, a, b) := | population_cod2Matrix ← decimalVector_to_cod2Matrix(population)
                         | transpond_population_cod2Matrix ← population_cod2Matrix^T
                         | rows ← rows(population_cod2Matrix)
                         | cols ← cols(population_cod2Matrix)
                         | p ← 1/cols
                         | for row ∈ 1..rows
                         |   out_row ← | if rnd(1) < p
                         |            |   | mutation_index ← floor(1 + rnd(cols))
                         |            |   | grey ← cod2_to_grey(transpond_population_cod2Matrix^⟨row⟩)
                         |            |   | grey_mutation_index ← | 0 if grey_mutation_index ≠ 0
                         |            |   |                       | 1 otherwise
                         |            |   | cod2_to_decimal(grey_to_cod2(grey))
                         |            | population_row otherwise
                         | for row ∈ 1..rows
                         |   | if out_row < a
                         |   |   | out_row ← a
                         |   |   | continue
                         |   | out ← b if out_row > b
                         | out
```

Рисунок 2.5 – Текст программы

Выполним элитарный отбор особей программой на рисунке 2.6.

```
elitar(f, parents, childrens, a, b) := | rows ← rows(parents)
                                       | fParents ← f(parents)
                                       | fChildrens ← f(childrens)
                                       | fParentsSoreted ← sort(fParents)
                                       | fChildrensSorted ← sort(fChildrens)
                                       | for row ∈ 1..round(rows/2)
                                       |   | firstRow ← row·2 − 1
                                       |   | secondRow ← mod[firstRow + 1, rows + (mod(rows + 1, 2))]
                                       |   | for elem ∈ 1..rows
                                       |   |   if fChildrens_elem = fChildrensSorted_row
                                       |   |     | out_firstRow ← childrens_elem
                                       |   |     | break
                                       |   | for elem ∈ 1..rows                    if secondRow > firstRow
                                       |   |   if fParents_elem = fParentsSoreted_row
                                       |   |     | out_secondRow ← parents_elem
                                       |   |     | break
                                       | out
```

Рисунок 2.6 – Текст программы

Далее вышеописанные функции были объединены в одну, для вычисления минимума, при помощи генетического алгоритма. Итоговый текст программы представлен на рисунке 2.7

```
genetycAlg(f, a, b, k, n, e) := | population ← init_population(a, b, k)
                                | for generation ∈ n
                                |   | weights ← weight(f, population)
                                |   | probabilities ← probability(weights)
                                |   | pull ← ruletka(probabilities)
                                |   | parents ← vibor(pull, population)
                                |   | newPopulation ← mut(crosingover(parents, a, b), a, b)
                                |   | population ← elitar(f, parents, newPopulation, a, b)
                                |   | minF ← min(f(population))
                                |   | break if minF ≤ e
                                | for pop ∈ 1..k
                                |   out ← population_pop if f(population_pop) = minF
                                | out
```

Рисунок 2.7 – Текст программы

Все выполненные расчеты представлены на рисунках 2.8-2.9.

$\mid\ \mid out \leftarrow b \ \text{ if } out_{row} > b$

$firstGeneration := init_population(a, b, populationSize) =$

	1
1	-19
2	31
3	9
4	-21
5	-14
6	21
7	-8
8	-17
9	-17
10	30

$func := myf(firstGeneration) =$

	1
1	723
2	$1.923 \cdot 10^3$
3	163
4	883
5	393
6	883
7	129
8	579
9	579
10	$1.801 \cdot 10^3$

$w := weight(myf, firstGeneration) =$

	1
1	0.09
2	0.239
3	0.02
4	0.11
5	0.049
6	0.11
7	0.016
8	0.072
9	0.072
10	0.224

$p := probability(w) =$

	1
1	9.386
2	5.258
3	14.813
4	8.497
5	11.97
6	8.497
7	15.351
8	10.362
9	10.362
10	5.504

$sumP := \sum p = 100$

$parentsPull := ruletka(p) =$

	1
1	1
2	3
3	1
4	7
5	5
6	1
7	7
8	7
9	6
10	1

$viborParents := vibor(parentsPull, firstGeneration) =$

	1
1	-19
2	9
3	-19
4	-8
5	-14
6	-19
7	-8
8	-8
9	21
10	-19

$crosingoverRes := crosingover(viborParents, a, b) =$

	1
1	-27
2	1
3	-18
4	-9
5	-30
6	-3
7	-8
8	-8
9	20
10	-18

$mutation := mut(crosingoverRes, a, b) =$

	1
1	-27
2	1
3	-18
4	-9
5	-30
6	-3
7	-8
8	-8
9	23
10	-17

$elitarResults := elitar(myf, viborParents, mutation, a, b) =$

	1
1	1
2	-8
3	-3
4	-8
5	-8
6	-8
7	-8
8	9
9	-9
10	-14

Рисунок 2.7 – Расчеты

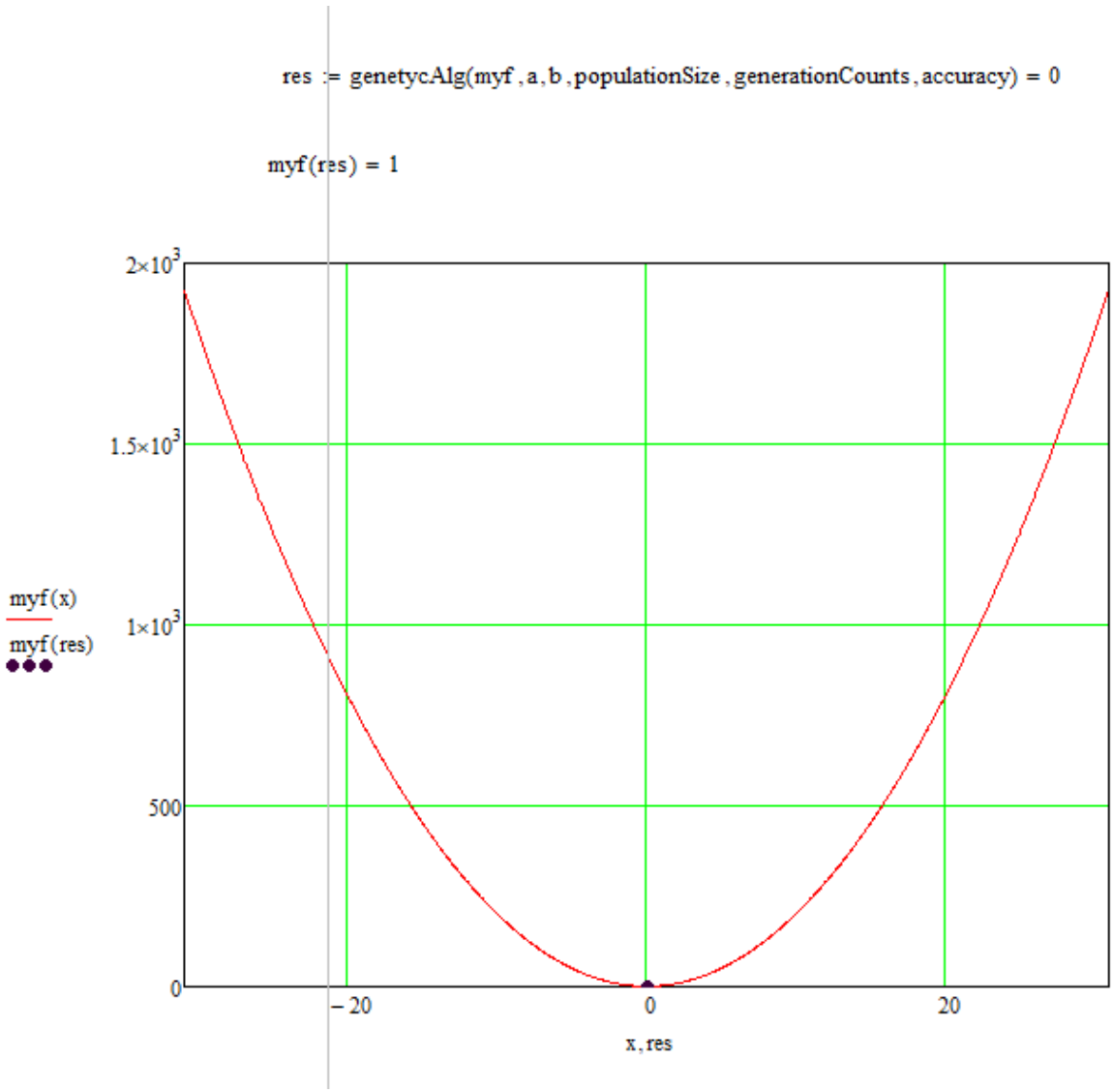

Рисунок 2.8 – Расчеты

Вывод

Во время выполнения лабораторной работы были получены навыки вычисления приближенного решения задач одномерной оптимизации на множестве целых чисел с помощью генетических алгоритмов.

Средствами Mathcad были написаны программы для кодирования и декодирования целых чисел с помощью двоично-десятичного кода.

Были написаны программы формирования начальной популяции, расчета фенотипов, расчета веса, выбора особей для скрещивания, оператора кроссинговера, мутации, элитарного отбора особей в новую популяцию.